

Project Proposal

The aim of this project is to create an application to generate travel routes in Lancaster, Preston, Blackpool and the Fylde and Wrye coast. This project will take existing data on travel methods including bus, rail, road and walking to create suitable routes from a start and end point input by a user.

This project will have two key aspects: the collection of start and end points and the display of generated routes. The collection of start and end points will be procured through either text or graphical input. Text based inputs will be collected through the use of two textboxes, one for the starting point and one for the final destination. Graphical inputs will be collected by the users dropping two pins onto a map of the area, the first pin to signify the start location and the second to signify the end location. Also present at this stage will be a display of the weather in user's current location to notify them of any potential travel disruptions.

The display of the generated routes will feature a map identifying the start and end points of the route, as well as highlighting the route itself. Information about each route will be present alongside the map, given an indication to the time each route will take. Additional information, such as the number of stops until the user's destination, will be available to the necessary routes. These generated routes will be able to be sorted according to a preference selected by the user. Weather information will again be present at this stage to provide an additional perspective to the user as to which the best route is for their day of journey.

Requirements

Software Requirements:

1. Functional requirements:

- 1.1. The application shall generate and show routes from a starting point to a destination for modes of transport including trains, buses, walking and driving.
- 1.2. The application shall be able to show routes that combine multiple modes of transport when necessary.
- 1.3. The application shall provide routes that cover the areas of Lancaster, Preston and the coastal regions of Blackpool, the Fylde and Wyre coast.
- 1.4. The application shall use weather conditions to recommend modes of transport.
- 1.5. The application shall allow users to filter their route based on speed, cost, and number of changes.
 - 1.5.1. The application shall allow user to set desired departure and/or arrival times.
- 1.6. The application shall allow user to create an account.
 - 1.6.1. The application shall be usable without an account.
 - 1.6.2. The application shall allow users with account to report traffic and incidents for other users.
 - 1.6.3. The application shall have loyalty program for users with account where frequent use can lead to rewards such as discounted tickets.
 - 1.6.4. The application shall allow user to save favourite routes.
- 1.7. The application shall include a map.
 - 1.7.1. The application shall highlight the routes on the map.
 - 1.7.2. The application shall allow users to input their start and end destinations by dropping pins onto a map.
- 1.8. The application shall show real time delays where applicable.

2. Non-functional requirements:

- 2.1. The application shall be supported on mobile and desktop devices.
- 2.2. The application shall generate at least 3 routes within 10 seconds of pressing 'find route' button.
- 2.3. The application shall have at least 2 sorting filters for the generated routes.
- 2.4. User account data shall be stored securely, following GDPR standards.
- 2.5. User passwords shall be stored in a hashed format.
- 2.6. The application should support 10,000 daily users at least.
- 2.7. The application shall be developed within the allotted budget.
- 2.8. The application shall be compatible with standard screen readers.

UI Mock-up

Figure 1 and 2: Sign in page/register page

Provides a clean and straight forward interface for users who wish to create an account or log in to access personalised features such as saved routes or check the number of points they have earned. The design prioritizes simplicity with clear labels for name (register page), email and password input fields. The design also provides a forgot password link for easy recovery while featuring a map in the background to aligns with the app's transport theme. Importantly authentication is optional, users can still explore core functionalities like route planning without signing in. The layout follows the basic accessibility principles with contrast in colour of the texts and background and large input areas and buttons.

Figure 3: Initial welcome map view

Serves as the app's welcome page, immediately showing the user and interactive map centered on their current location which is indicated by a blue dot. A search bar is placed at the top allowing a user to enter a destination right away. While the bottom tool bar icons allow users to drop a pin as their destination or origin, filter their routes based on specific preferences, and a navigation button that allows user to centralise the map with their current location.

Figure 4 and 5: Searching and route suggestions

The route suggestion screen displays a scrollable list of suggested routes after a user has entered a destination on the search bar, presenting a minimum of 3 alternative routes with each option showing the key details like Estimated Time of Arrival (ETA) and number of changes. The suggestions are laid on top of the map allowing users to compare routes efficiently while maintaining visual context of the city. The filter button at the bottom of the screen allows users to customize preferences while the weather icon button on the right side of the screen allows users to check the weather in their desired destination or origin.

Figure 6 and 7: Weather page and Filter page

The weather details triggered by tapping the weather icon provides a contextual weather information tailored to the user's current location, destination and the journey. The conditions are displayed with text alongside the relevant icons for better accessibility. With the use of the weather, the app also highlights a route recommendation purely based on the weather, encouraging a more enjoyable travel. The filter page offers users control over the

main display and route planning results, divided into three clear sections. Buttons are presented as large, easy to tap switches with descriptive labels.

Figure 8: Timing selection page

Appears after selecting a destination, allowing users to specify whether to depart now, depart at a certain time in future or arrive by a specific time. The search bar at the top remains for quick editing.

Figure 9: Route visualisation screen

The app highlights the route options directly on the map as well as the textual presentation, allowing an instant visual comparison. A small non interactive label also appears next to each route, differentiating the options while preserving map readability.

Figure 10: Dropping pin option

Allows users to manually place a pin on the map, to set a precise destination, origin or stop. The tool bar at the bottom of the screen is for action buttons like save, direction and other actions. The design maintains minimal overlaying ensuring the pin dropping process feels intuitive.

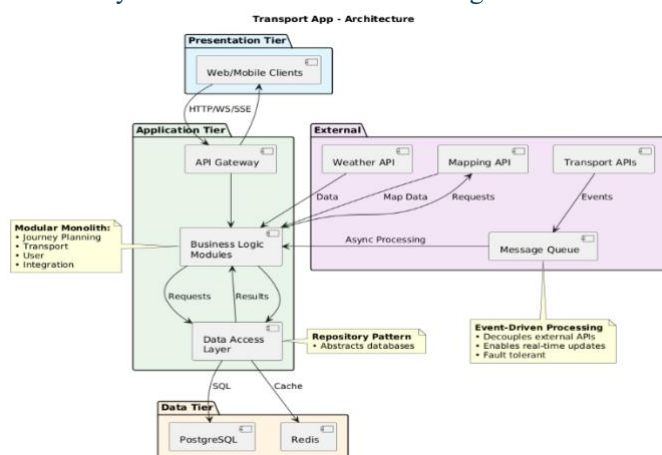
Figure 11: Active navigation screen

Presents the selected route with a clear highlighted path, showing the journey and the necessary stops along the way. On the top, a concise instruction panel displays the next action to take with a more button which displays the whole journey's actions if pressed. The report button on the bottom left allows user to submit a real time report on traffic, accidents, roadwork or other issues to benefit the community.

Figure 12 and 13: Profile page and setting

The profile displays the key account information and the point balance; in the middle there is a clear expandable section that lists redeemable discounts or rewards. The setting button provides access to usual account preferences like device and network settings with large tappable buttons, clear labels and clear contrast in colour for accessibility.

System Architecture / Code Design



Architectural Design

Overview

Our architecture adopts a hybrid N-tier approach that combines client-server, modular monolith, and event driven patterns (Neal, et al., 2022). Rather than adhering to one singular architectural style we have employed a pragmatic, problem driven methodology where patterns are selected and composed based on their ability to address our specific challenges (Martin, 2003).

The fundamental pattern our architecture is built upon is a client-server model which naturally matches users request-response interactions our system will be dealing with. Our architecture has 3 physically separated Tiers:

- **Presentation Tier** – web and mobile clients
- **Application Tier** – business logic & domain services data access code
- **Data Tier** – Persistent storage & caching

Within the Application Layer we have opted for Modular Monolithic organization, as it offers clear separation of concern and team parallelism without the operational complexity of distributed microservices (S., 2015). Event driven processing for real-time transport data ingestion and live updates has been incorporated using a message

queue to decouple data ingestion from user requests. While not a tier, external service integration has been considered, and implemented using adapter patterns allowing for third-party transport, weather, and mapping services/data. External transport data will be ingested from the provided feeds: static timetables are preloaded into PostgreSQL; real-time bus positions are pulled via REST; live rail messages are consumed via STOMP message queue and processed asynchronously.

Client-server Foundation & Rationale

The basic idea of a client-server is that a client request and servers provide services (Hassan, 2011). Within the project, the client is a user facing application delivered as an interactive web application. This client constitutes the presentation layer in our tiered design and is responsible for all user interaction and interface rendering. Following this, the server is the factory that fulfils that client's requests and encompasses the Application Tier (node.js monolith) and the Data Tier (PostgreSQL and Redis containers).

We adopted the client-server model as our foundational architecture because it directly addresses some of the core demands of our application. Firstly, it enables the centralisation of complex logic and data aggregation, which is not only important for combining different external data feeds (bus, rail weather) but computing multi-modal routes on adequate server hardware, rather than burdening users' devices. Secondly, it provides inherent security and data integrity due to the confinement of sensitive operations and database access to a controlled server environment. Thirdly, the separation of client/server supports independent scalability and maintainability as well as centralising updates to logic and APIs without needing client-side patches. Finally, it facilitates efficient parallel development by allowing for a clear contract (API) between back end and front-end teams.

While alternatives like microservices offered superior independent scalability/flexibility, they would have added significant operational overhead to development. An example of this is having to manage distributed data consistency which is not a problem for client-server architectures. For a small development team, with a fixed deadline, this added complexity may have jeopardised our delivery without any real benefit. Alternatively, we adopted a modular monolith within our client-server foundation, to provide necessary separation of concerns for parallel development with less operational overhead.

Use Cases



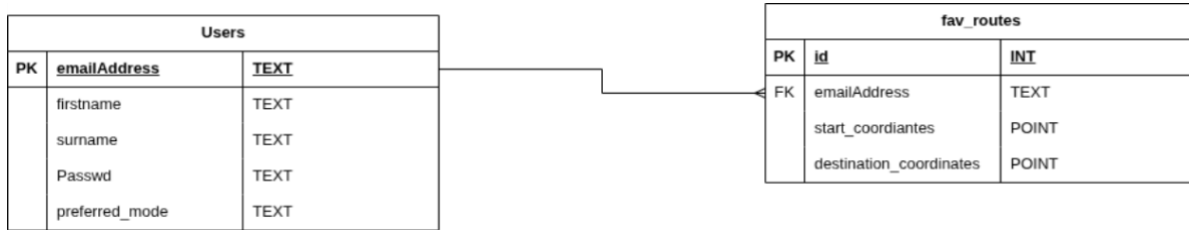
This diagram illustrates how a user will be able to interact with the application. The main ways that a user can interact with the application are planning a route, selecting a filter to sort the generated routes, logging in with or without an account, reporting traffic and saving a route. Each time the user logs into the application, the credentials they entered will be checked against those stored in the user database to determine whether the entered credentials are correct. Each time the user reports traffic along the route they are taking, the transport database will be updated so that the times of other routes generated for other user are more accurate. When a user plans a route, they are required to provide three more pieces of information, which are the start location, the end destination and either the arrival time or the departure time.

This diagram also illustrates how a developer is able to interact with the application after deployment. The main ways that a developer can interact with the application are updating the loyalty points on a user account, updating the map and updating the transport and user databases. Each time a developer updates either the transport or user database, data will be modified leading to the databases becoming actors in the diagram.

Database Design

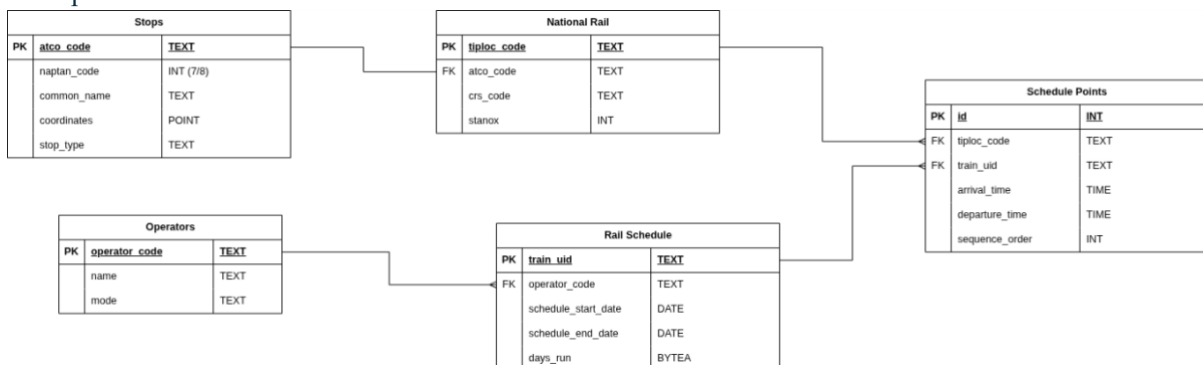
We are going to use a relational database because it will allow us to store data from the many files. We only need to parse the data once and then we will be able to make queries to retrieve or update data in specified areas. We have chosen to use PostgreSQL due to its ability to handle large datasets. MySQL was also considered however; PostgreSQL allows for more flexibility with different data types such as point.

Users and Saved Routes



This will store user account information; passwords will be in a hashed format. We will make sure that any data kept about the user will be handled in accordance with the Data Protection Act (DPA) 2018. We will also have a table holding the user's favourite routes. This will have their email address, and coordinates for start and destination. We will use the point datatype which comes with PostgreSQL as this allows us to calculate distances to other coordinates. Storing the email address multiple times will lead to the redundancy, however, since it is not a lot of data, it will be quicker to simply store the email address multiple times instead of querying multiple tables.

Transportation



These tables will be essential for collating all the information from the multiple datasets. As different operators and modes of transport use different methods of storing information, we have designed this to streamline transport information. This will allow us to run queries to find train times for certain platforms and platforms. It also allows us to translate the different codes which different datasets use into their 'common name' which will be displayed to the user on the application. All this information is public; no hashing is needed.

Algorithms

Finding Rail Routes

```
route_calculation_train(startPoint, endPoint):
    Create array to store the train companies that go to the station closest to start point
    Create an array to store the train companies that go to the station closest to the end point
    Create an array to store the stations that will be part of the final route

    Find all train companies that go to the starting station and store them in the correct array
    Find all train companies that go to that end and store them in the correct array

    Compare the train company arrays:
    If a matching train company is found:
        Store all the lines that the matching train operators use at the start station
        direct_train_route(lines[], endPointTrainOperators[])
    If no matching train company is found:
        indirect_train_route(lines[], endPointTrainOperators[]):
            For each line:
                For the next station along the current line:
                    Collect the train operators that go to the current station
                    Compare these train operators with the train operators that go to the end station
                    If there is now a matching train operator:
                        Update the lines array to store the line with a matching train operator
                        direct_train_route(lines[], endPointTrainOperators[])
                    If there is not a matching train operator:
                        If the current station == the last station along the line:
                            If the current line != the last line:
                                Look at the next line
                            If the current line == the last line:
                                Return an empty array as the final route
                                Break the loop
                        If the current station != the last station along the line:
                            Look at the next station

direct_train_route(lines[], endPointTrainOperators[]):
    For each line:
        Look at each stop on the line
        Record the stop
        If the current stop == end station:
            Return the series of stops
            Break the loop
        If current stop != end station:
            If the index < the length of the stations array:
                Increment the index of the stations array to look at the next station
            If the index >= the length of the stations array:
                If the index for the line array < length of the line array:
                    Increment the index for the line array to look at the stations along the next line
                If the index of the line array >= the length of the line array:
                    indirect_train_route(lines[], endPointTrainOperators[])
```

This algorithm will generate the route between two train stations. This is achieved by comparing the train companies that go to the start and end stations on the route to determine whether a direct train route can be found. Matching train companies at both the start and end stations will indicate that a direct route can be found, so will lead to the direct route algorithm being called. In this algorithm, each stop along each line at

the start station that had a train operator that matched a train operator at the end station will be recorded and examined to see if it is the user's desired final train station. Until the final station has been reached, this algorithm will continue to loop. If during the search for the final station the current station is the end station on a particular line, but not the desired end station, then the next line will be examined until the desired end station is found. If all

stops along all the lines have been examined and the desired end station has not been found, then an indirect route will be searched for.

The indirect route algorithm will be called when no matching train operators have been found between the start and end stations or when a direct route has failed to be generated. In this scenario, each line that has the start station as a stop along it will be examined. The train operators that make journeys from each station along each line will be collected. These train operators will then be compared against the train operators of the user's desired end station. If there is now a matching train operator, then the lines array will be updated to store the current line that has a train operator that matches with the train operators at the end station and the direct route array will be called to begin the calculation of a direct route from this point. If there is still not a matching train operator, then the next station along the current line will be examined until there are no more stops along this line. If all stops along all the lines have been examined and there are no matching train operators at any of them, then an empty array will be returned to indicate that it is not possible to generate a route by train.

Bus Routes

```
route_calculation_bus(startPoint, endPoint):
    Create array to store the bus companies that go to the stop closest to start point
    Create an array to store the bus companies that go to the stop closest to the end point
    Create an array to store the stops that will be part of the final route

    Find all bus companies that go to the starting stop and store them in the correct array
    Find all bus companies that go to the end stop and store them in the correct array

    Compare the bus company arrays:
        If a matching bus company is found:
            Store all the lines that the matching bus companies use at the starting bus stop
            direct_bus_route(lines[], endPointBusCompanies[])
        If no matching bus company is found:
            Find all the lines that go to the starting bus stop
            indirect_bus_route(lines[], endPointBusCompanies[])

indirect_bus_route(lines[], endPointBusCompanies[]):
    For each line:
        For the next stop along the current line:
            Collect the bus companies that go to the current station
            Compare these bus companies with the bus companies that go to the end stop
            If there is now a matching bus company:
                Update the lines array to store the line with a matching bus company
                direct_bus_route(lines[], endPointBusCompanies[])
            If there is not a matching bus company:
                If the current stop == the last stop along the line:
                    If the current line != the last line:
                        Look at the next line
                    If the current line == the last line:
                        Return an empty array as the final route
                        Break the loop
                If the current stop != the last stop along the line:
                    Look at the next stop

direct_bus_route(lines[], endPointBusCompanies[]):
    For each line:
        Look at each stop on the line
        Record the stop
        If the current stop = end stop:
            Return the series of stops
            Break the loop
        If current stop is != end stop:
            If the index < the length of the stops array:
                Increment the index of the stops array to look at the next stop
            If the index >= the length of the stops array:
                If the index for the line array < length of the line array:
                    Increment the index for the line array to look at the stops along the next line
                If the index of the line array >= the length of the line array:
                    indirect_bus_route(lines[], endPointBusCompanies[])
```

These algorithms will generate a route between two bus stations. The main route calculation algorithm will begin by finding the bus companies that go to the starting and end stops. These bus companies will then be compared to determine if there is a bus company that takes a route that involves both the start and end stops. If so, then a direct bus route will be looked for by calling the `direct_bus_route` algorithm. This will involve every stop on every

bus line from the starting stop that uses the same bus company that goes to the ending bus stop being examined. Each bus stop will be recorded so that a possible route can be created. If the bus stop that is currently being examined is the end stop, then the loop will be broken and the route that has been determined will be returned. If not, then this algorithm will continue to loop until every stop on every line has been examined and if none of these stops are the user's desired end stop, then an indirect route will be looked for.

When the `indirect_bus_route` algorithm is called, all stops along all the bus lines will have the bus companies that go to a particular stop collected and compared against the bus companies that go to the end stop. If at a bus stop, there is a match in bus companies, then a direct bus route from that stop will be looked for. If there is still no matching bus company, then the algorithm will continue to loop through the remaining stops on all the remaining lines. If every stop on every line has been examined and no matching bus companies have been found, then a bus route between the start and end stops the user entered cannot be calculated.

Finding Walking Routes

```
route_calculation_walk(startPoint, endPoint):
    Create an array to store the streets for a route
    Create a variable to store the current street
    Add the street that the start point is situated on to the route array
    While the street that the end point is situated on has not been reached:
        Follow the current street in the direction towards the end point
        If a junction is reached:
            Update the current street
            If the current street is not the same as the starting point street:
                Add the current street to the route array
        If a corner is reached:
            Update the current street
        If the current street is equal to the end point street:
            Break the loop
    Return the route array
```

This algorithm will produce a route for a user to follow by walking from a start point to an end point input by the user. This is done by finding the street that corresponds to the user's inputs for both the start and end locations. Once this has been achieved, the street that the starting point is situated on is added to an array to store the route that can get the user to their desired destination. The current street is then followed until a junction or a corner is

reached, at which point the current street will be updated to reflect the street that the algorithm is currently following to find a route. This process will continue until the current street is the same as the end street, in which case the loop will break and the array that holds the streets that the algorithm has traversed will be returned.

Finding Driving Routes

```
route_calculation_drive(startPoint, endPoint):
    Create an array to store the roads for a route
    Create a variable to store the current road
    Add the road that the start point is situated on to the route array
    While the road that the end point is situated on has not been reached:
        Follow the current road in the direction towards the end point
        If a junction is reached:
            Update the current road
            If the current road is not the same as the starting point street:
                Add the current street to the route array
        If a corner is reached:
            Update the current road
        If the current road is equal to the end point street
            Break the loop
    Return the route array
```

point is situated on to determine whether a route has been created. If whilst following the current road a junction or a corner has been reached, then the current road will be updated to reflect the change in the road, if applicable. Each time a new road is encountered, it will be added to the final route array to keep track of the roads that the algorithm has traversed. This process will continue until the current road is the end road, at which point the final route array will be returned.

Combining Multiple Routes

```
route_generation(startPoint, endPoint):
    currentStartPoint
    currentEndPoint

    route1[] = route_calculation_walk(startPoint, endPoint)
    route1TransportMethods = ["Walking"]

    route2 = []
    route2TransportMethods = []

    currentEndPoint = Coordinate of the train station closest to the startPoint
    route2WalkToStation[] = route_calculation_walk(startPoint, currentEndPoint)

    currentStartPoint = currentEndPoint
    currentEndPoint = Coordinate of the train station closest to the user end point
    route2Train[] = route_calculation_train(currentStartPoint, currentEndPoint)

    currentStartPoint = currentEndPoint
    route2WalkFromStation[] = route_calculation_walk(currentStartPoint, endPoint)

    If route2WalkToStation[] is not null:
        Append "Walking" to route2TransportMethods
        Append each element of route2WalkToStation to route2
    If route2Train[] is not null:
        Append "Train" to route2TransportMethods
        Append each element of route2Train to route2
    If route2WalkFromStation[] is not null:
        route3[]
        route3TransportMethods = []

        currentEndPoint = Coordinate of the bus stop closest to the startPoint
        route3WalkToStop[] = route_calculation_walk(startPoint, currentEndPoint)

        currentStartPoint = currentEndPoint
        currentEndPoint = Coordinate of the bus stop closest to user end point
        route3Bus[] = route_calculation_bus(currentStartPoint, currentEndPoint)

        currentStartPoint = currentEndPoint
        route3WalkFromStop[] = route_calculation_walk(currentStartPoint, endPoint)

        If route3WalkToStop[] is not null:
            Append "Walking" to route3TransportMethods
            Append each element of route3WalkToStop to route3
        If route3Bus[] is not null:
            Append "Bus" to route3TransportMethods
            Append each element of route3Bus to route3
        If route3WalkFromStop[] is not null:
            Append "Walking" to route3TransportMethods
            Append each element of route3WalkFromStop to route3

    route4[] = route_calculation_drive(startPoint, endPoint)
    route4TransportMethods = ["Driving"]
```

calculates a train route is called. The current end point is then also updated to the coordinates of the train station that is closest to the end location input by the user. These two updated variables are then the parameters when the train route calculation algorithm is called as the current start and end points are now both train stations. Finally, the current start point is updated again to reflect the final train station on the train route. The walking route calculation algorithm is then called once more to calculate the potential final walk to the final destination. The arrays that store the routes from each of the function calls are then examined. If the contents of an array is not empty then that route is appended onto the entire route array, and the mode of transport used is added to the transport type array as the modes of transport required for a particular route will be displayed to user along with the route itself. If the contents of any of the arrays is empty, then that particular array is discarded as it indicates that a route was not able to be formed between the start and end points.

The third route that the algorithm looks at is the route for taking the bus and, potentially, walking. This structure is the same as the second algorithm, and the same steps will be followed, however, the algorithms for calculating the bus route will be called instead of those for calculating a train route.

Finally, the algorithm looks at calculating the route that involves just driving. In this case, the algorithm calculates the driving route will be called using the inputs provided by the user as parameters. The route output by this

This algorithm will produce a route for a user to follow by driving from the start point to the end point input by the user. The algorithm will begin by creating an array to store the roads that the algorithm follows from the starting road to the road that the end point is situated on. At the beginning of the algorithm, the current road will be equal to the road that the start point is situated on. The current road will then be compared against the road that the end

This algorithm makes use of all the previous algorithms to calculate both simple and combination routes. The four routes that are highlighted in this algorithm are: just walking, taking the train and walking, taking the bus and walking and just driving. Along with each route, there is also an array that contains each transport method that is required for a particular route to be achieved.

This algorithm begins with generating a route that just involves walking. In this case, the algorithm for calculating a route for walking will be called using the inputs from the user as the parameters and its output will be stored in the route1 array. As this method only involves walking, the transport methods array for this route will only contain 'Walking'.

The second route that the algorithm looks at is the route for taking the train and, potentially, walking. Before this route is calculated, empty arrays for the route itself and the transport methods required for this route are set up. The current end point is then updated to reflect the coordinates of the train station that is closest to the starting location entered by the user and the walking route calculation algorithm is called with the user entered start point and the current end point as parameters to the algorithm. Once this algorithm has been completed, the current start point is updated to reflect the current end point for when the algorithm that

algorithm will be stored in the array route4. As this method only involves driving, the transport methods array for this route will only contain 'Driving'.

Design Principles

Overview

Our architecture has been guided by the following core design principles, and have directly influenced technical decisions, module organization and implementation methods. These principles help guarantee our system is maintainable, scalable and meets both user needs and project constraints. Input validation, fault tolerance, and user-centred design also informed our decisions, the four principles below had the most direct impact on our architectural structure

Separation of concerns

This principle ensures each component has a single well-defined responsibility/use. This design principle has been demonstrated within the architectural decision for a n-tier structure, that provides clear distinct boundaries between responsibilities. Furthermore, there is also a deeper separation of concerns within the Application tier with the use of independent modules (e.g., Journey Planning, Transport). This separation is beneficial, as it allows for parallel development, simplified testing and changes are contained within their well-defined areas. This however has more performance overhead and additional layers may introduce latency affecting user experience, but this can be mitigated through caching and optimization.

Modularity

This principle dictates a system should be composed of discrete, interchangeable components that encapsulate a specific functionality. This is demonstrated within our Application Tier through the adoption of a modular monolith, where independent modules (e.g., journey planning) each shall reside in their own directories with explicit public interfaces. Modules communicate through well-defined contracts rather than direct implementation access. External services are wrapped in adapter modules to isolate third-party dependencies from the system; this is beneficial as it allows for future replacement/modification without system-wide effects.

Loose Coupling

This principle ensures that components have minimal knowledge of each other's internal implementations and dependencies are kept to a minimum. This design principles are demonstrated by:

- Message queue: decouples external transport APIs from internal business logic
- The Repository Pattern: Isolates database operations behind abstract interfaces
- API Gateway: prevents client directly accessing backend services

This decoupling is beneficial as it improves fault tolerance and maintainability. Admittedly this adds complexity to communications patterns and may require extra error handling logic, but this can be solved through standardized adapter interfaces and/or circuit breaker patterns

Asynchronous Processing

The principle ensures that long running or non-blocking operations do not impede user interaction with the system. This principle was demonstrated through our event driven architecture, where live transport data is ingested via a message queue and processed independently of HTTP request cycles (Gregor & Woolf, 2003). WebSocket and SSE connections provide real-time updates to clients without polling, and complex multi-modal route calculations can return preliminary results while refining options in the background. This non-blocking approach is beneficial to maintaining UI responsiveness during intensive processes and efficiently handles concurrent user connections. This increases complexity in state management and requires race conditions to be handled carefully. Well defined event schemas and idempotent processing patterns can help mitigate these trade-offs.

Project Plan for Implementation

Milestones and deliverables

The group has key dates and deadlines that they are working towards, each week builds upon previous weeks so that we are constantly evolving the application into different stages. These milestones include what we hope to have achieved by the end of that week and what we could provide back to the stakeholders of the application.

Week 16 (Friday 20th of February):

This week is one of the most important weeks this sets the baseline for the application. Understanding what the stakeholders want and require for the application to work (functional and non-functional), establishing chain of commands, identifying risk we may encounter, resource planning and understanding and setting up Agentic AI services for the software development.

Week 17 (Friday 27th of February):

This is where we implement our design from our UI mock-ups for our front end of the application. This is making the dashboard for the application and laying out what the application should look like. We implement live demonstrations to the customer about how our application will look.

Week 18 –19 (Friday 6th – 13th of March):

These two weeks are providing live demonstrations the stakeholders such as integrating APIs and automation for the application, implementing security controls. These weeks prove to our stakeholder that we are providing on our promises and pushing forward towards the deadlines. Week 19 we will also be submitting ethics forms for the next week where we will be user testing the application.

Week 20 (Friday 20th of March):

The development team want to run user evaluation on this week. This is a part of our quality assurance process to make sure that our application is useable and accessible for our users. These acid tests will determine whether our application is functional for our users.

Week Easter period (Friday 27th of March – Friday 24th of April):

These weeks will be used to test functionality and improve the application. Such as making the application more multiplatform and accessible to users. This would also be when we check where our testcases are passing and if any changes are needed to be made.

Week 21 (Friday 24th of April):

Formal deadline to deliver the application to the client is set at Friday the 24th of April this means handing over all code and the architecture of the application, so users can setup and launch the application for people to use. After this week we are contractually obliged to finish all work unless deadlines are pushed back.

Quality assurance

The other style of quality assurance that we will be using is acid testing. This would be performed by stakeholders (mainly users) where paid testers would simulate tasks users would partake in when on the application.

Developers will have no partake in this testing to ensure the bias is kept to a minimum as they would understand how the application would run meaning they understand how features work and how they will be displayed. This type of testing would be used simulate initial users of the system and test if features such as the destination or live weather updates are user friendly.

Regression testing is used to improve upon, and fix identified issues to meet user requirements, fix issues with the system and improve security for the users. This is important as flaws that are pointed out need to be addressed, such as having security issues about how we store user data can lead to serious issues. This means that algorithms used to hash out user data or SQL injection protocols to cleanse inputs need to be tested and improved if there is an issue. Additionally, these patch releases or updates to the system can break previously working infrastructure for the system. Testcases are also being used to see whether the concurrent features that are working together are interacting correctly with each other to build a suitable outcome. Developers build test case scenarios to check features such as the login, computing the routes and checking whether their favoured routes take them to the correct location.

Other forms of testing will be used such as heterogenetic testing to check if our application is multiplatform for all users of different operating systems. Whilst also conducting stress testing to see if our application can support our users while under heavy load to make sure the system and APIs are responding correct when put under heavy load. Overall, quality assurance is vital to maintain and upgrade existing systems so that are users are kept satisfied and the application is running successfully.

Role's responsibilities and governance

Roles are imperative to maintain a function project run. One developer was chosen by the group to be appointed as the project/team lead. Having a team lead that governs roles and task delegation minimises confusion and streamlines decision making.

All developers are obligated to build testcases so that any code that is pushed onto main has scenarios and working services. This is because nonfunctioning API services can lead to faulty data on the interface, these situations need to be kept at a minimum. Every developer is responsible for all code that is pushed to our repository, meaning that unverified code that could be plagiarised, direct upload from Agent AI or using AI platforms that are not permitted, that person is accountable for.

Besides from internal governance we are also governed by legislations that govern data collection. GDPR and data protection states we need to make sure that data is securely stored, and we have sufficient reasoning for the collection of personal identifiable data. This is enforced by the ICO meaning that we must adhere to the rules otherwise we will be sanctioned. Besides from data collection we need to make sure that we also adhere to PCI if the application collects payments from our user. These legislations are mandatory to think about when building the software

Risk analysis of the tasks in project plan

This is a risk analysis of the project; we have highlighted concerns and provided mitigation tactics for the risk. The RAG scale ranks the likelihood of the issue and the impact on the project from 1 to 5, then subsequently the likelihood and impact are multiplied to get a quantified outcome to confirm its RAG status.

PROBABILITY	VERY HIGH	5	10	15	20	25
	HIGH	4	8	12	16	20
	MEDIUM	3	6	9	12	15
	LOW	2	4	6	8	10
	VERY LOW	1	2	3	4	5
RAG Matrix		VERY LOW	LOW	MEDIUM	HIGH	VERY HIGH
		IMPACT				
KEY:		RED	AMBER		GREEN	

Cost risk: RAG status [amber]

One of the main concerns that we have as a group is overrunning our budget. This could lead to serious issues such as funding disputes within the group or the need to ask stakeholders (investors) for additional funding to make sure the project is delivered at a quality and timeframe that is agreed on. This can have a high (4) impact on aspects of the team, such as our reputation as we would be seen as an organisation that can't estimate a price for a project and maintenance. This could also lead to the group having to make out of pocket spending to make sure that critical services that are needed for the project to run are in place. This issue has a low (2) likelihood of occurring, as we have made sure all scenarios are monitored and all costs such as maintenance are fixed.

This is rated at an **amber** category threat as if overspending was to occur it can have lasting effect and cause a pipeline of issues. We are making sure all cost and expenditure is audited by the team lead biweekly to make sure that no unnecessary spending is in place. We are also keeping in touch with investors and clients to make sure that in the event we do overspend they are informed and made clear of why and what we want to overcome the issue.

Scope Creep: RAG status [amber]

Control of the scope and associated delivery of outcomes needs to be central to enable project sustainability. As a group we have made sure the project delivers well within its financial resource allocations and especially within the client's vision of what the application should be. The group may not have clearly identified what the client wants, meaning they may overshoot or undershoot features necessary to the client's wants and needs. This is a medium (3) probability as stakeholder may withdraw or add features to the application which wasn't explicitly stated to team previously. This is medium (3) impact on the team could be immense; it could add additional costs

which we didn't forecast or push deadlines we have set. This is an **amber** RAG status meaning developers on this project need to be aware about the issue and need to actively avoid it.

Mitigation protocols such as MoSCoW prioritisation is used to analyse the spectrum of what features are needed, e.g. must have, should have, won't have. This helps our developers know what our client wants so we can implement features without any roadblock and with clear understanding.

Meeting Deadlines: RAG status [**amber**]

Team members falling ill or absent may cause pushback on internal deadlines or not meeting the final production date to deliver the application to the client. Pushing the deadline back has an impact on both the development team and the client. The client expects the project to be delivered within the timeframe. Developers may have differing skills gaps, commitments with other projects/needs; these shortfalls need to be made up for. The potential impact is rated at a medium (3). This issue has a high (4) likelihood of occurring; this is because a person may have external commitments delaying the work or is finding a certain section of the application difficult to produce, meaning they may need assistance for them to reach their deadlines. Alongside this, we cannot guarantee people won't fall ill.

This is an **amber** level threat; we are confident that people working on certain tasks for this project can complete their assignment. As a team we have planning in place for all developers to make sure they maintain the next step is. However, due to it being a scenario that is probable to occur and with significant impacts on the group the rating was necessary. As a group we have made sure that we have clear communication. This is through using different communication services such as Microsoft Teams, WhatsApp group chats and emails. This further allows us to clear up any issues to stay on track. We have also added productivity support such as Gantt charts that allow everyone in the group to understand when deadlines are due. Additionally, activity network diagrams are being used to assist in project management as it shows the developer bottlenecks and potential delays they may encounter when building the application allowing them to work around it. Furthermore, with Agentic AI we can reduce the amount of time taken on their section due to the additional support it provides meaning remedial tasks can be completed.

Agentic AI code: RAG status [**red**]

Agentic AI may generate hallucinations that compound over time. This is known as cascading errors, leading to issues such as a user facing data or faulty interfaces for the app. Additionally, this leads to insecure or non-compliant code into the application, which is a major issue as incorrect or unreliable outputs can lead to functional failures. This may result in increased workload for the project or a security issue exposing user data or other security flaws. This can degrade the quality of the work we produce alongside building vulnerabilities for users making the project not meet compliance/regulatory standards. The impact on the application is high (4) because of how much it can deviate the project.

This has a high (4) likelihood of occurring as Agentic AI services such as Copilot Pro will be at the forefront of most code creation. This is a **red** level category with as issues if unchecked and untested can have devastating problems if integrated into the application. As a group, we have adopted a zero-trust stance on code generated by the AI. This means that all team members will have to verify all code output that they generate alongside double checking to make sure it doesn't relate to any merge conflicts that may occur. These checks are done to make sure that all outputs are secure and good. These errors may be hard to reverse when in later stages of the design. Alongside the manual checks, we are restricting the AI so that there is no sensitive or user data in the prompts to make sure that all user data remains confidential. Alongside this, besides looking at outputs, we are making sure that the outputs in the chat are reviewed to minimise the AI from going on a tangent.

Paid testers: RAG status [**green**]

Testers may provide unrepresentative or low-quality feedback on our test it means we may miss aspects such as accessibility or useability factors on our application, limiting our audience. The impact of the issue is medium (3), this is because accessibility features are already embedded into devices such as screen readers, or viewing settings such as screen size is already adjustable to the user without the use of our application. This would improve the ease of use of the application, but developers need to make sure that the application adopts these features. This

issue has a low (2) likelihood of occurring as there are multiple testers with differing background and accessibility limitations. We have rated this problem **green** as the development team recognises that there is an issue. However, with diverse testers and user acceptance testing, the model scenarios ensure the system works and key features are tested to ensure quality. We are hosting a range of testers to make sure all corners of the application are tested for ease of use and accessibility. Testers are completing a checklist to make sure all scenarios are covered.

Conclusion

Overall, the risks are highlighted for everyone in the group to be made aware of and methods to alleviate those risks from occurring. The biggest threat of Agentic AI has been assured to all group members about stances they should take when using the service. Some threats are not treated equally such as paid testers; this is because we assume that the likelihood of occurrence is low and impact is minimal. Everyone is accountable for their actions in using mitigation tactics to avoid risks, this is to make sure that the project is ran smoothly and without errors. All work will be audited by group members and the team leader so that production is straightforward and no issues have gone under the radar. Any new threats that may arise when building the project will be highlighted initially to the team leader and subsequently strategies will be produced so that the team can combat the issue.

Costings

Developer costs:

10 week project time, 15 hours a week per developer

150 hours per developer total

£60 hourly rate * 150 hours * 5 developers = £45,000

Project lead + developer: £100 hourly rate

£100 * 150 hours = £15,000

For an expected 10,000 daily users at peak in the initial release of the app, £5,000 per year for hosting.

£650 per week for office space needed totals to £6,500 for the whole time the project is running.

£20 an hour to employ external testers, budget for testers is £2,000

GitHub Copilot Enterprise at £39 per user per month

£39 * 6 * 3 = £702

Maintenance guarantee for 5 years: £30,000

This cost is for fixing any bugs that occur, maintaining the servers and making general updates. Any new features are not covered under maintenance.

Total Cost: £104,202

Task List

Category	Tasks
UI	- Create a UI to input starting and ending locations and allow setting the arriving and/or departing time, defaulting to current time if unused. - Display all the routes. - Have a map displayed as the main point on computers. On mobile, it should be a separate UI to the selection UI. - Add an option to drag and drop a pin to select starting / ending locations instead of inputting with text. - Implement a feature on the map that highlights the top 3 routes in different colours with indication of any mode of transport changes. - Use the timings to display the estimated time that routes will take and have the departure times from useful intervals on the route (start, any changeovers). - Display weather on the requested route. - Create a UI to let users create an account but also allow users to use without creating an account. - Let users report traffic and road incidents and show these to users whose routes are affected. - Implement a feature to allow users to favourite a route.
Database	- Create a database to store the user account details. Passwords to be hashed before being stored, need to implement a hash function. - Store favourited routes in the database - Store traffic and road incident reports in the database
Other	- Create initial web app. - Implement an algorithm to find multiple routes between requested points, allowing the use of any mode of transport. - Implement a filter and sorting system to disregard routes that don't fit their filter and order the routes according to how a user wants them presented. The filter system should consider time, cost and the most direct route.

- Have feeds from <http://transport.scc.lancs.ac.uk/> to get data about timings.
- Read from an API to estimate any traffic for delays.
- Have weather forecasts/current weather conditions read from an API.

Acceptance Testing

Multi-Modal Transport Options

Scenario	Inputs	Expected Outcome
Planning trip from Point A to B using map pin	Given starting location coordinated and destination location coordinates from user	Return at least 3 routes, at least one should integrate multiple modes of transport
Planning a trip from Point A to B using typed locations	Given text input for a start location and destination location	Translate these inputs into co-ordinates as fitting to typed location as possible. The system should return at least 3 routes, at least one should integrate multiple modes of transport.
Route Selection based upon filters and preferences	Given start and destination coordinates AND Given filter options (such as no railways or maximum changeovers) Or user preferred mode of transport saved on their account	Return multiple routes integrating transport modes while conforming to the filters.
Route selection with adverse weather conditions	Given start and destination coordinates and, Given weather conditions at these coordinates which would affect certain modes of transport (e.g. icy roads)	Return multiple routes using multiple modes of transport, while selecting recommended travel options based on weather conditions. (Recommending trains if road is icy)

Live bus updates

Scenario	Inputs	Expected Outcome
Live bus location update	Given the user selects a certain bus route on the map Obtain live update on bus location from the API	Add bus icon on map to where the live bus co-ordinates are. Should update every 15 seconds.
Bus delay / timetable change	Given a bus is delayed i.e. live location is different to where it should be according to timetable	Application should update the expected arrival at next stops.

Rail Service

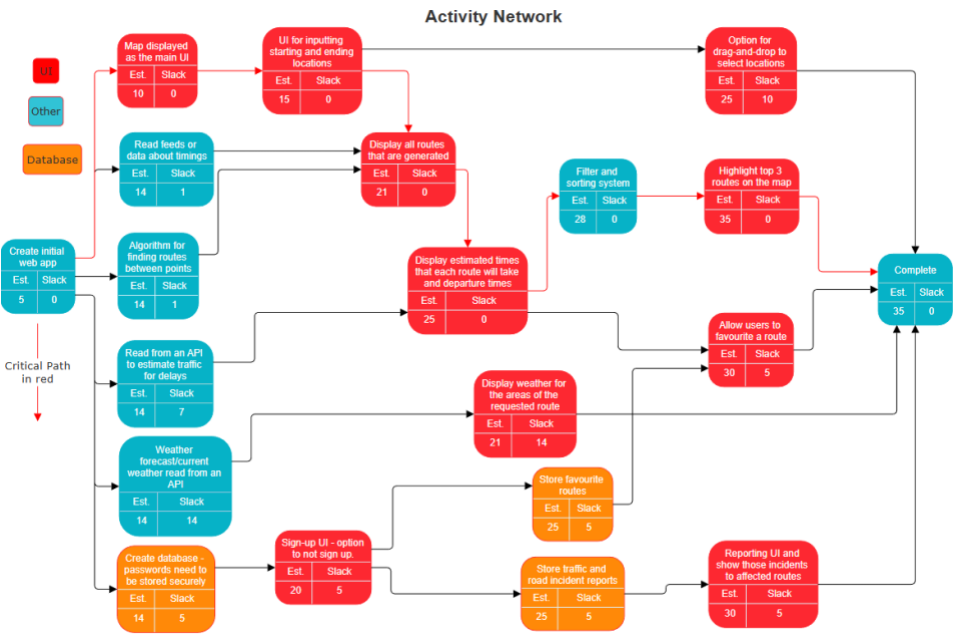
Scenario	Inputs	Expected Outcome
Train live location updates	Given a user has selected a certain route including a train and, That specific train is moving between berths	Update the train's location to the location of the berth which has been passed Display this on their user map with a train icon.
User wants to view departures from a particular station	Given the user has selected a route which involves train travel	Display other departures from the station which will take them to the desired destination.

User Accounts

Scenario	Inputs	Expected Outcome
New user trying to register an account	Given a user's email address, password, first name and surname entered via register page.	Hash this password using secure hashing algorithm If they are a new user, store their details in user table. Return successfully created account message and return to login page.
User trying to register account with already used email address	Given a user's email address, password, first name and surname entered via register page.	If their email address matches with one already in the database Return error message saying email already in use.
Current user trying to login	Given a user's email address and password entered via login page	If their email address and password hash matches one found in database, then give them access to their account Take to home page
Attempting to login with either invalid email address or invalid password or both	Given a user's email address and password entered via login page	If their email address or password hash does not match one in the database, then display "invalid account details" error message on login page.

Activity Network

This activity network shows what tasks we must do and an estimated time it will take us to complete them. The red arrows between tasks shows the critical path – the tasks that must stay on schedule or we will fall behind in how long our estimated completion time is. The tasks with black arrows have slack time – the amount of time we can fall off schedule without pushing back the completion time. A lot of this slack time is shared over many different tasks.



Gantt Chart

This Gantt chart shows the same as the activity network above, but in a chart format instead of a network of nodes. It makes it easier to visualise how much slack time we have on certain tasks and will help us to prioritise tasks that are more urgent even if they aren't critical – those with lower slack times. Tasks like the weather-related ones are a lot less urgent and therefore don't need to be as focused on as others.

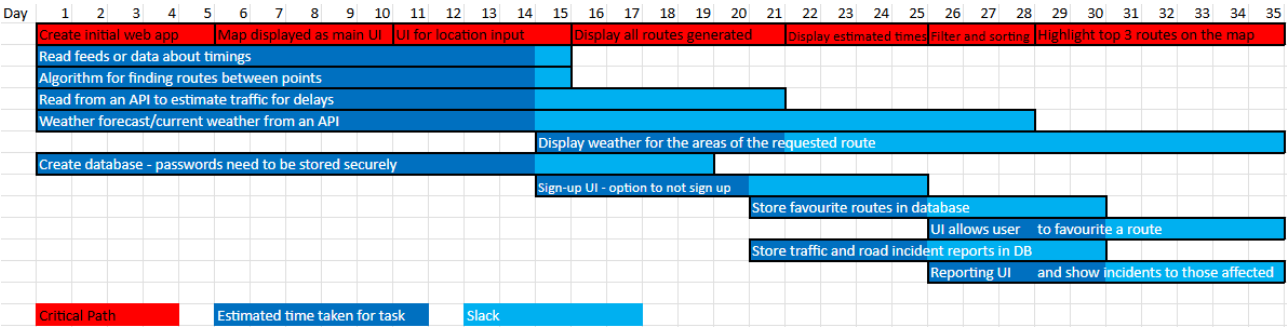


Figure 1

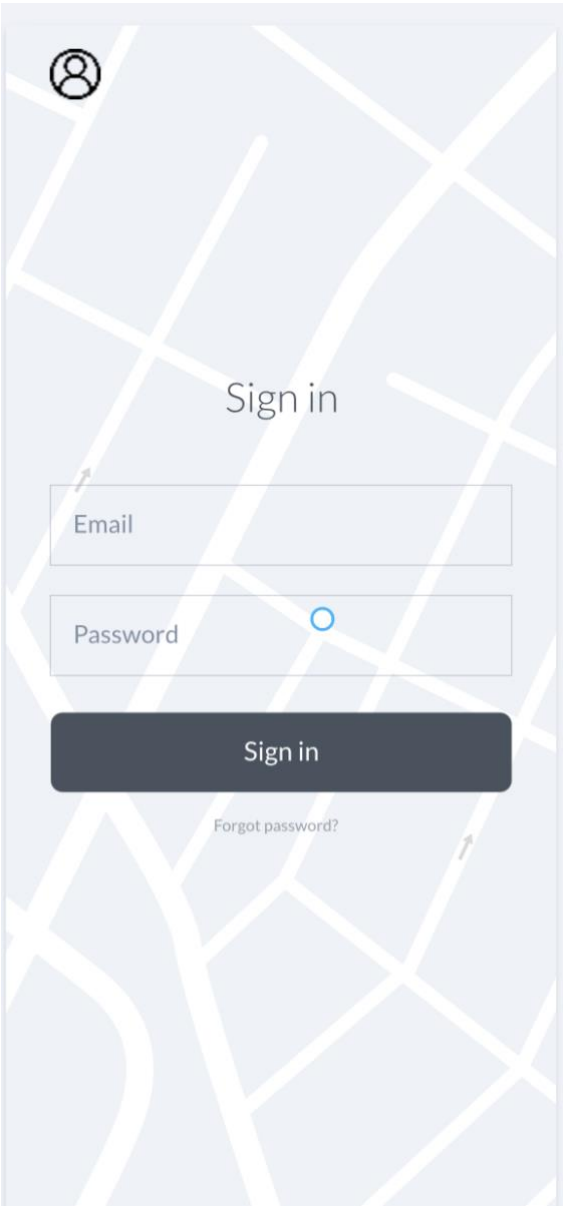


Figure 2

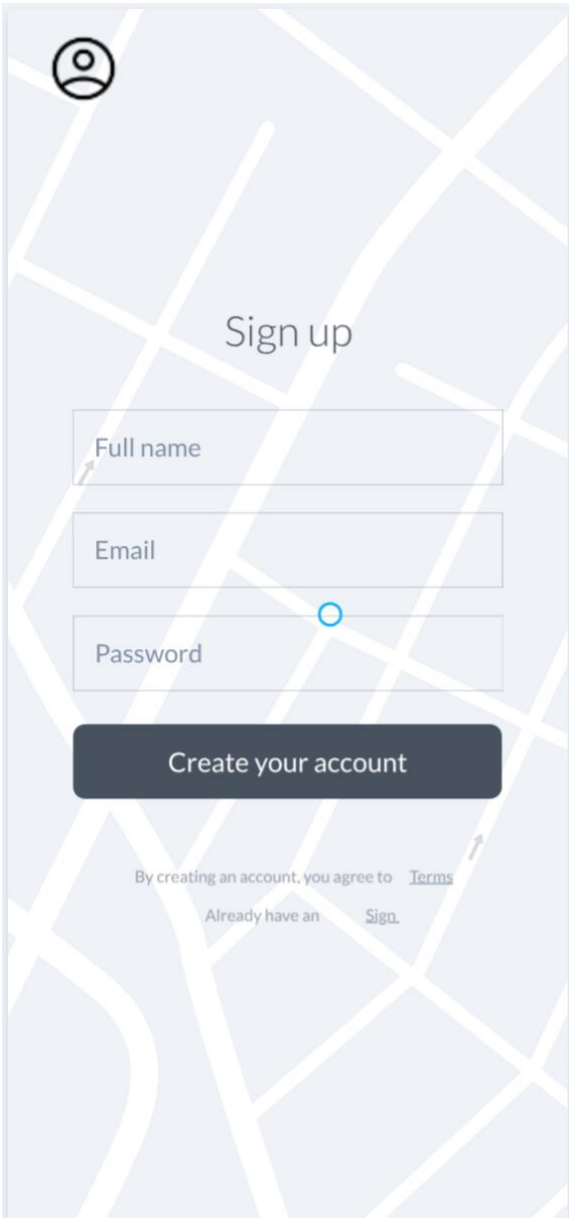


Figure 3

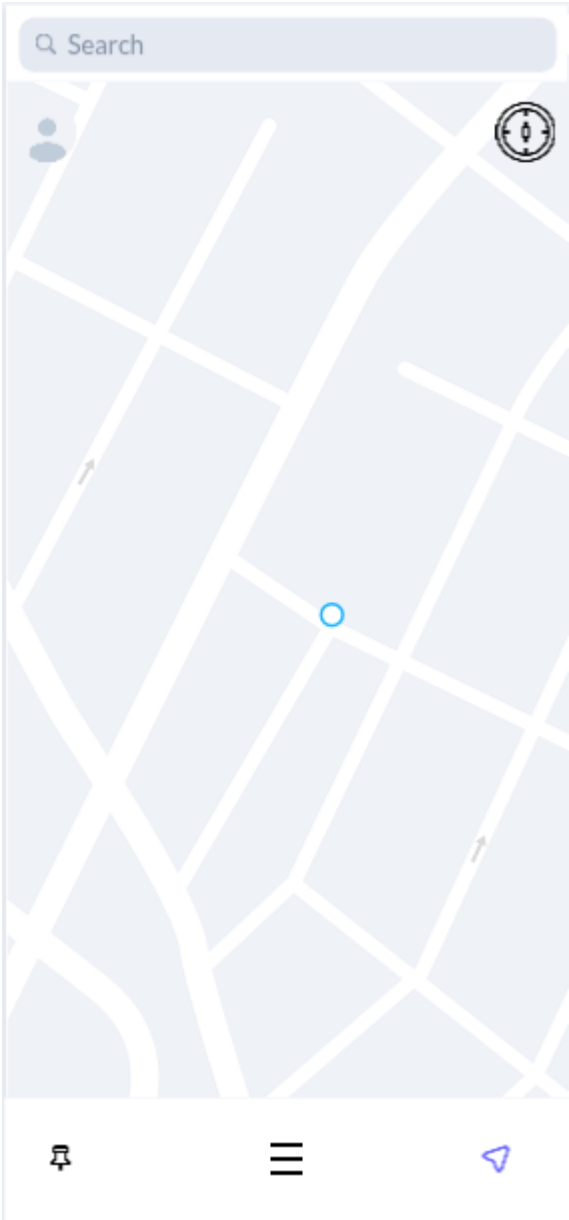


Figure 4

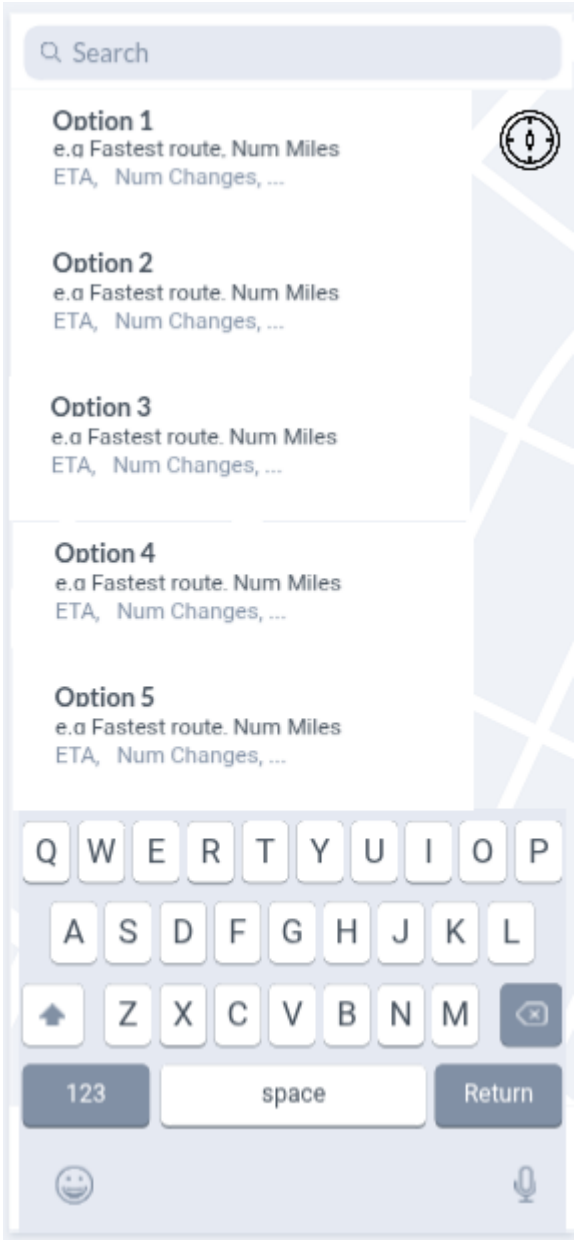


Figure 5

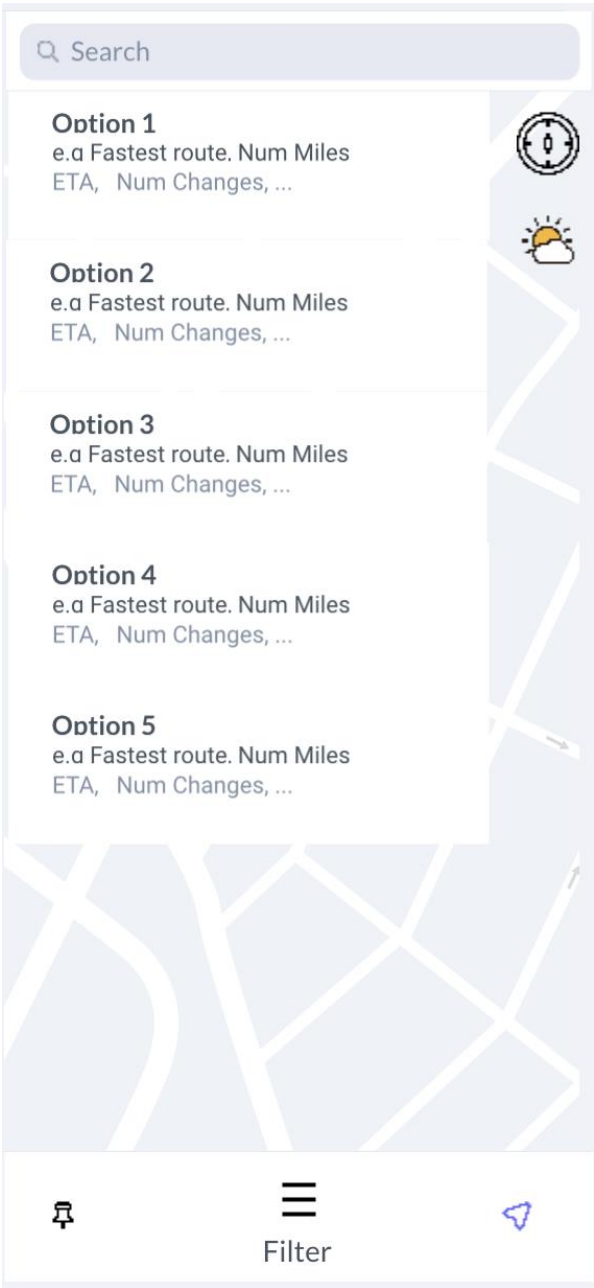


Figure 6

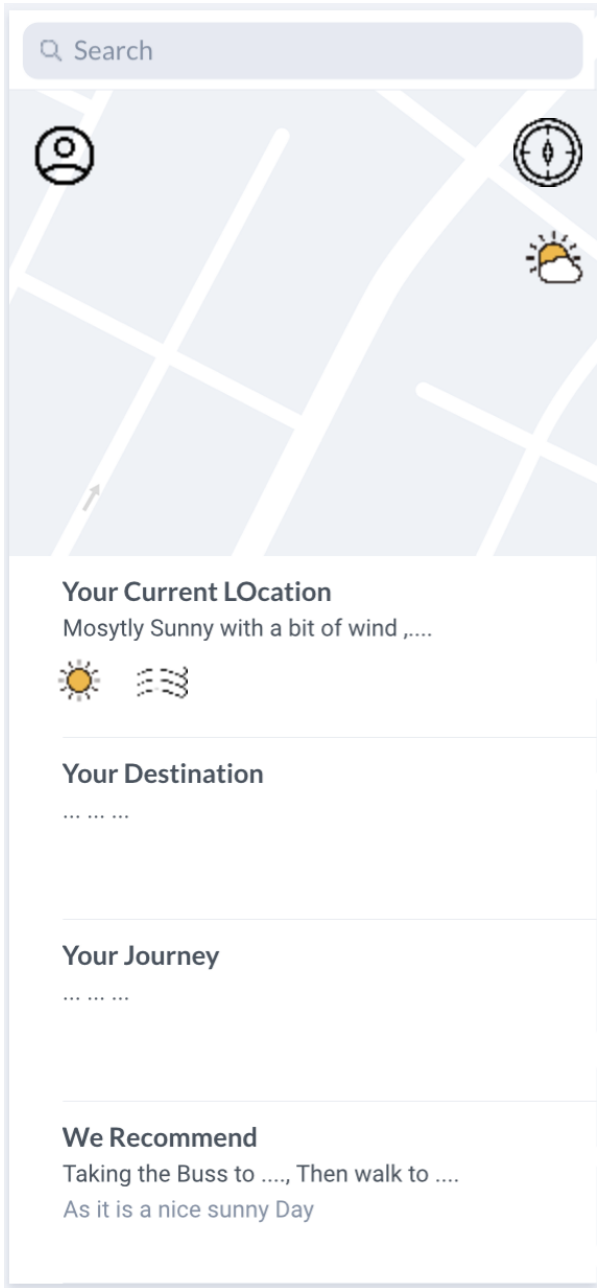


Figure 7

Back

Filter

ON MAP

Show Bus Stops

Show Train Stops

Show Traffic Condition

...

...

DIRECTIONS

Include Walking

Include Driving

Include Taking The Bus

Include Taking The Train

...

ROUTE SUGESSTION

Fastest

Cheapest

Least Num Changes

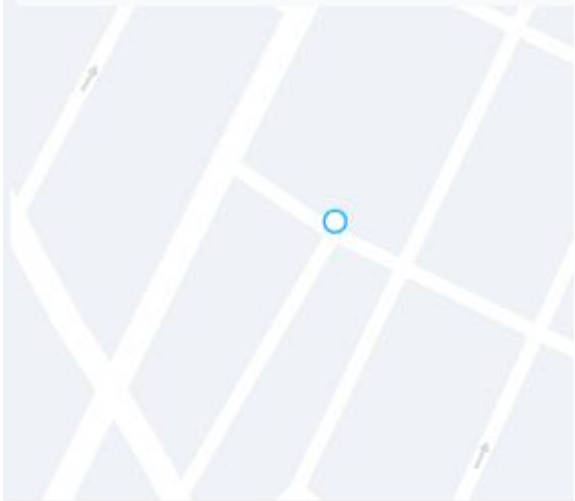
Figure 8

Search

Leave now

Depart at

Arrive by



Sun 6 Sep757

Mon 7 Sep858

Tue 8 Sep959

Wed 9 Sep1000

Thu 10 Sep1101

Fri 11 Sep1202

Sat 12 Sep1303

Figure 9



Figure 10

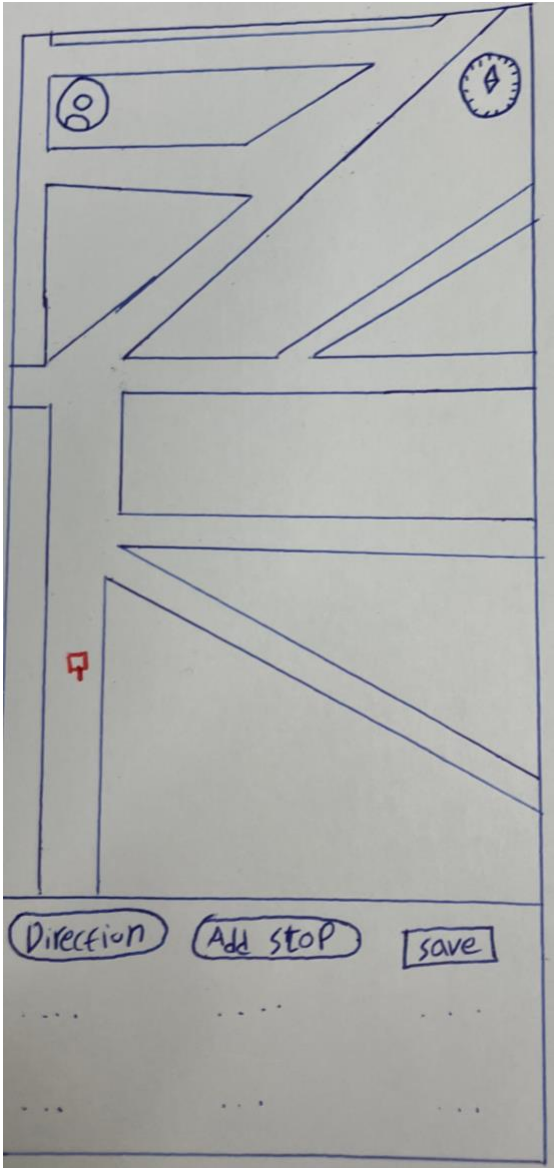


Figure 11

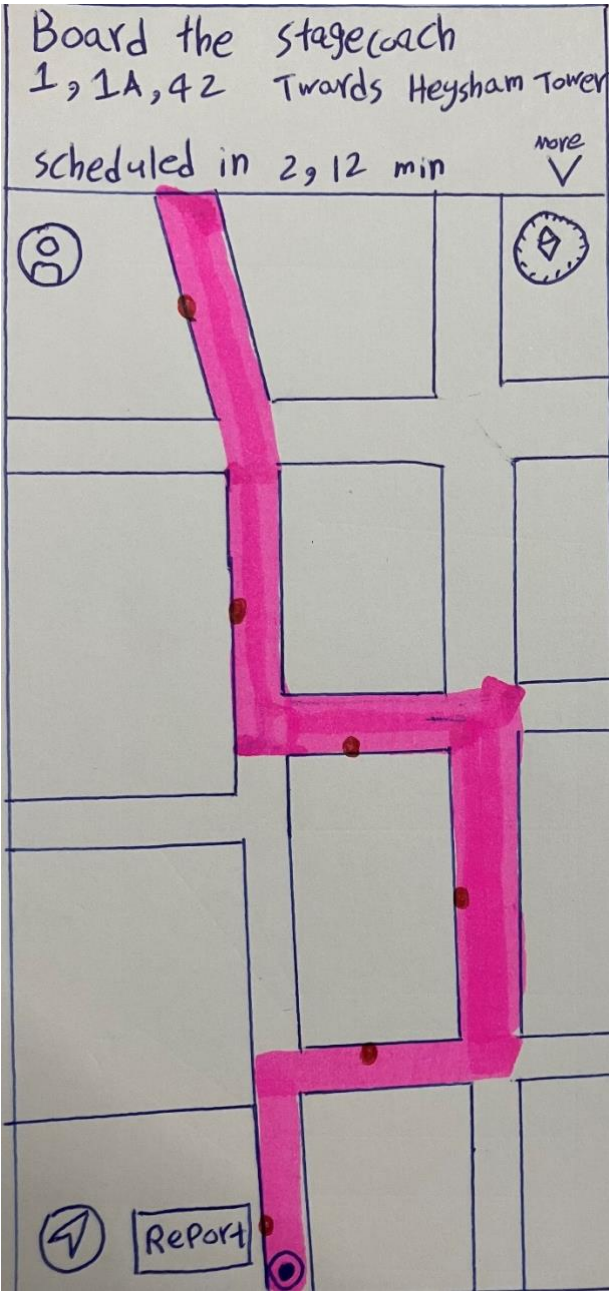


Figure 12

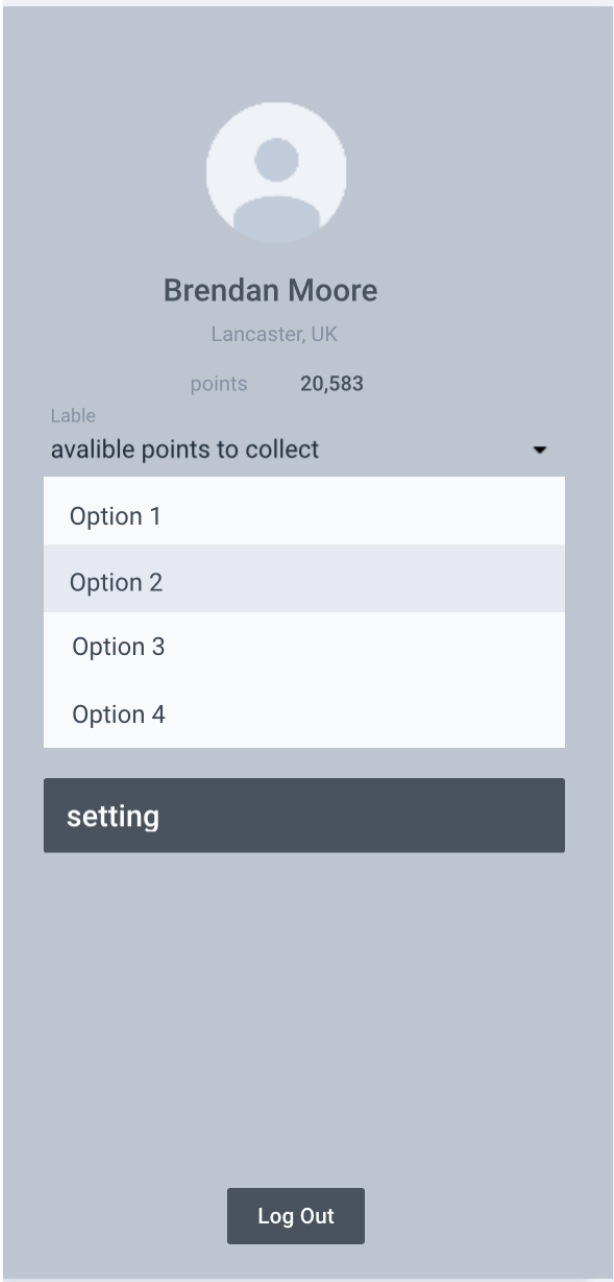
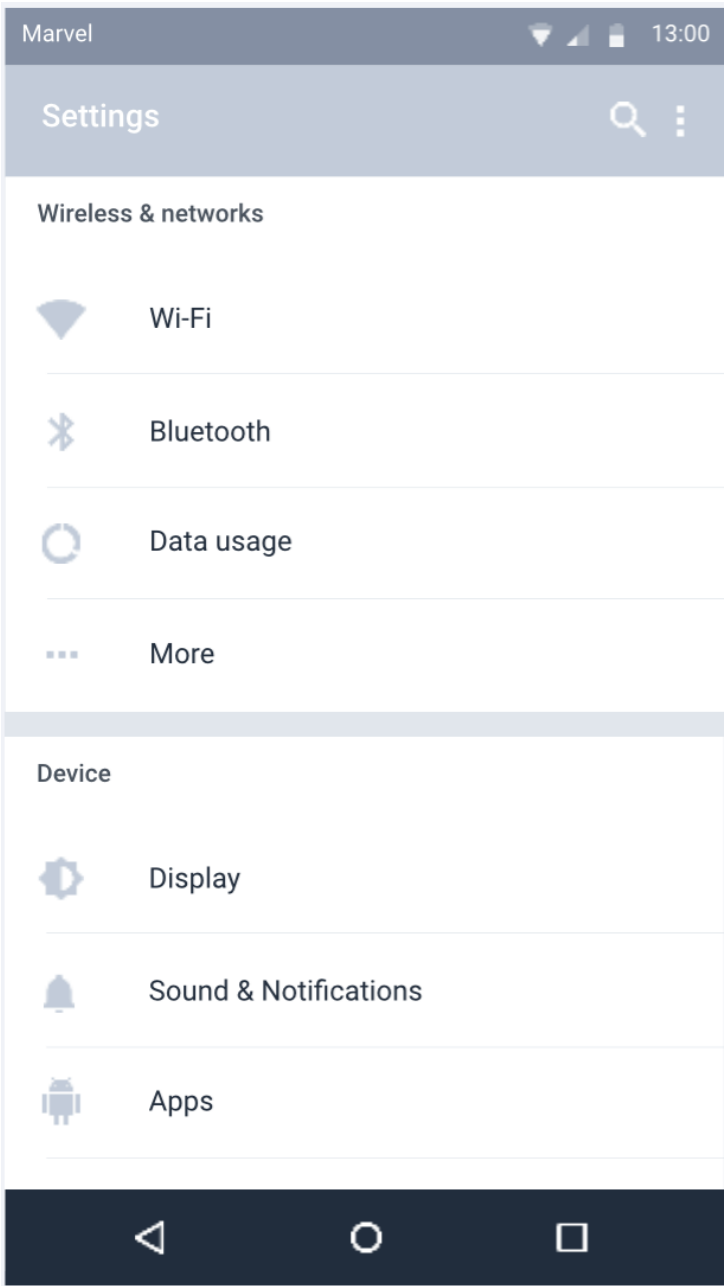


Figure 13



AI Use Declaration

Section AI has been used for	AI tool used	How the tool was used	How the output was verified
NA	NA	NA	NA

We declare that we have used Artificial Intelligence tools only as described above. All submitted work reflects my own understanding, and I take full responsibility for its accuracy and integrity.

References:

Soler, J. (2025). Zero-Trust Security for Generative AI. NeuralTrust. Accessed: [Monday 26th of January 2026]. Link: <https://neuraltrust.ai/blog/zero-trust-security-for-generative-ai>

ADMIN (2026). Cascading Failures in Agentic AI: The Definitive OWASP Security Guide. Adversa AI. Accessed: [Monday 26th of January 2026]. Link: <https://adversa.ai/blog/cascading-failures-in-agentic-ai-complete-owasp-asi08-security-guide-2026/>

Heine, C. (2022). What is scope creep and how can we mitigate it? www.apm.org.uk. Accessed: [Monday 2nd of February 2026]. Link: <https://www.apm.org.uk/blog/what-is-scope-creep-and-how-can-we-mitigate-it/>.

Ashton, D. (2025). What is scope creep (and how can you prevent it)? University of the Built Environment. Accessed: [Monday 2nd of February 2026]. Link: <https://www.ube.ac.uk/whats-happening/articles/scope-creep/>.

Gregor, H. & Woolf, B., 2003. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. 1st ed. Boston: Addison-Wesley Professional.

Hassan, G., 2011. 'Designing Client/Server Software ARchitectures', in *Software Modelling and Deisgn: UML, Use cases, Patterns, and Software ARchitecture*. Cambirdge: Cambridge University Press.

Martin, F., 2003. *Patterns of enterprise application architecture*. Boston: Addison-Wesley Professional.

Neal, F., Rebecca, P. & Patrick, K., 2022. *Building Evolutionary Architectures*. 2nd ed. Sebastopol: CA: O'Reilly Media.

S., N., 2015. *Building Microservices: Designing Fine-Grained Systems*. 1st ed. Sebastopol: CA: O'Reilly Media.